ChatGPT

State-Synchronization Protocol for Video & Sandbox

To synchronize video playback with an interactive sandbox, we will use xAPI statements for telemetry, an **iframe+postMessage** scheme for sandbox isolation, and a **Web Worker** for fast input validation. Each custom event (e.g. `initialized_sandbox`, `submitted_payload`, `resolved_error`) is logged as an xAPI statement (actor, verb, object, context). We build on the xAPI Video Profile specification: for example, video events use verbs like `played` (`https://w3id.org/xapi/video/verbs/played`) and set `context.contextActivities.category.id = https://w3id.org/xapi/video` 【49†L83-L90】【49†L111-L118】. We do likewise for sandbox events. For instance, we might define a verb IRI `https://example.com/xapi/verbs/initialized_sandbox` with display "initialized sandbox", and send a statement with:

```
{
  "actor":
{"objectType":"Agent","name":"Alice","mbox":"mailto:alice@example.com"},
  "verb": {"id":"https://example.com/xapi/verbs/initialized_sandbox","display":
{"en-US":"initialized sandbox"}},
  "timestamp": "2026-09-22T10:00:00Z",
  "object": {
    "objectType": "Activity",
    "id": "https://example.com/sandbox/12345",
    "definition": {
      "type": "https://w3id.org/xapi/video/activity-type/video",
      "name": {"en-US":"Interactive Video Sandbox"},
      "description": {"en-US":"Sandbox for coding exercise synced to video"}}
  },
  "context": {
    "contextActivities": {"category":[{"id":"https://w3id.org/xapi/video"}]},
    "extensions": {
      "https://w3id.org/xapi/video/extensions/session-id": "session-1234",
      "https://w3id.org/xapi/video/extensions/time": 12.34  // current video
time in seconds
    }
  }
}
```

In practice, we use the video profile's extensions for synchronization: for example, `result.extensions.time` holds the video timestamp【49†L105-L107】, and `session-id` ties together events【49†L118-L121】. Custom sandbox verbs follow the same pattern: include a clear IRI and English display, an object (the sandbox or payload), and use the video profile category to indicate the video

context【49†L111-L118】. This way all telemetry – video actions and sandbox interactions – are unified under xAPI.

Embedding and postMessage IPC

We embed the interactive sandbox (e.g. an n8n flow, Monaco editor, React Flow app) inside an HTML `<iframe>` to isolate its state. The parent page and iframe use the browser's `postMessage` API for low-latency IPC. By design, `window.postMessage()` is a built-in asynchronous channel for cross-window messaging【52†L198-L206】. The data passed is serialized with the structured-clone algorithm, so we can safely transmit complex objects (JSON, arrays, etc.) without manual stringification【56†L243-L247】. In our setup, the **parent page** listens for messages from the iframe (`window.addEventListener('message', …)`) and vice versa. Each message should include a clear `type` field (e.g. `"init"`, `"submit"`, `"error"`) and any payload data. For example:

```
// In parent page:
window.addEventListener('message', event => {
  if (event.origin !== "https://oursite.com") return;  // verify source
  const { type, data } = event.data;
  if (type === 'initialized_sandbox') {
    // handle sandbox init
    logXAPIEvent('initialized_sandbox', data);
  }
  // ... handle submitted_payload, resolved_error, etc.
});
```

```
// In sandbox iframe (child):
window.parent.postMessage({ type: 'initialized_sandbox', data: { /* details
*/ } }, "https://oursite.com");
```

Low latency: Under normal conditions, `postMessage()` is very fast (sub-millisecond) between same-origin frames. To ensure we don't flood the channel if events fire rapidly, we use *debouncing*. As one developer notes, if many messages are triggered in quick succession, it's best to delay (e.g. 10–50ms) before sending and reset the timer on new events【54†L139-L146】. This avoids UI thrashing while keeping latency under ~50ms. For example, if the sandbox is sending state updates on every keystroke or drag event, we would accumulate changes and send one message after a short delay.

Security: We strictly set the `targetOrigin` (not "*") on every `postMessage` to avoid leakage【56†L250-L258】. Likewise, the receiver always checks `event.origin` before processing【56†L343-L347】. MDN warns that "a malicious site can send a message to any window" in the same iframe hierarchy, so verifying `origin` (and even message format) is critical【56†L343-L347】. In our case, the sandbox iframe is controlled by our code (and marked same-origin with `sandbox allow-same-origin allow-scripts` if using the `sandbox` attribute), so these checks confirm we only act on known messages. Overall, the parent and iframe communicate directly through fast `postMessage()` events, ensuring interactive sync (playhead position, user actions, etc.) with typical latency well below 50ms.

Feedback Verification with Web Workers

To validate student input without blocking the UI, we offload checks to a **Web Worker**. A worker runs in a separate thread, so heavy computation (like JSON-schema validation or code execution checks) won't stall rendering【66†L244-L248】. We spin up a dedicated worker (`new Worker("validator.js")`) when the exercise loads. The main page and worker communicate via `postMessage` / `onmessage`: e.g., when the student submits their code/payload, the parent does `worker.postMessage(payload)`. Inside the worker, an `onmessage` handler parses and validates the payload (e.g. using a lightweight schema check or custom logic), then `postMessage({valid: true, errors: [...]})` back to the main thread.

MDN notes that data between a worker and the main thread is passed by copy (not shared)【66†L244-L248】. In practice this means the JSON or object we send is cloned, avoiding race conditions. A minimal example setup:

```
// main.js (UI thread)
const validator = new Worker('validator.js');
validator.onmessage = e => {
  const { valid, errors } = e.data;
  if (valid) {
    displaySuccess();
  } else {
    displayErrors(errors);
  }
};
function onSubmit(payload) {
  validator.postMessage(payload);  // send data to worker for checking
}
```

```
// validator.js (Web Worker thread)
onmessage = function(e) {
  const payload = e.data;
  let valid = true, errors = [];
  // Example: simple JSON-schema-like check
  try {
    const obj = JSON.parse(payload);  // if expecting JSON string
    if (!obj.answer) {
      valid = false;
      errors.push("Missing 'answer' field");
    }
    // ... other lightweight checks ...
  } catch(err) {
    valid = false;
    errors.push("Invalid JSON");
  }
```

```
  postMessage({ valid, errors });  // send result back to main thread
};
```

Because the worker runs independently, even a worst-case validation (within reason) finishes quickly (<<500ms) without freezing the UI. We keep the checks minimal (e.g. no blocking loops) so response is instant. This pattern – main page sends data, worker validates, then main receives result – is standard for headless evaluation【66†L244-L248】. (More complex setups might load a compiled AJV schema or similar inside the worker for robust JSON validation.) The key is that **every student submission triggers** `worker.postMessage()`, and the UI listens for `validator.onmessage`, so validation happens asynchronously.

By combining these pieces – xAPI statements for telemetry, iframe+postMessage for sandbox isolation, and a Web Worker for fast checks – we achieve a synchronized "game loop" of video and code exercise. In summary:

- **Telemetry:** Define xAPI verbs for sandbox events, attach them to the video context (using `category: https://w3id.org/xapi/video`【49†L111-L118】 and extensions like `time` 【49†L105-L107】).
- **Synchronization:** Embed the interactive environment in an iframe and communicate state (timestamps, events) via `window.postMessage()`, following best practices (structured clone for data, debouncing, strict origin checks【56†L243-L247】【54†L139-L146】【56†L343-L347】).
- **Feedback:** Use a Web Worker to "headlessly" verify user code/payload on submit, sending results back so the UI can respond immediately without lag【66†L244-L248】.

This approach ensures tight sync between video progress and the live sandbox, reliable telemetry logging, and snappy feedback loops (all well under 50ms latency for messaging and under 500ms for validation).

Sources: The xAPI Video Profile specification provides templates and rules for video-related statements (verb IDs, context categories, extensions)【49†L83-L90】【49†L111-L118】. MDN documentation and community Q&A explain how `postMessage()` enables fast iframe communication and the importance of debouncing and origin-checking【52†L198-L206】【54†L139-L146】【56†L343-L347】. Web Workers are likewise documented as a way to run background scripts with `postMessage/onmessage` messaging (data is cloned)【66†L244-L248】. These materials guided the protocol design.
